

**LAPORAN PRAKTIKUM**  
**PEMOGRAMAN BERORIENTASI OBJEK**  
**“CLASS, OBJECT, CONSTRUCTOR DAN METHOD**  
**Interaksi Objek & Manipulasi State (Mini Bank)”**



**disusun Oleh:**  
**NAIRA RAMADHANI HALIL**  
**2511533027**

**Dosen Pengampu:**  
**NURFIAH, S.ST., M.Kom.**

**Asisten Praktikum:**  
**Siti Fadhilah Rahmi**

**DEPARTEMEN INFORMATIKA**  
**FAKULTAS TEKNOLOGI INFORMASI**  
**UNIVERSITAS ANDALAS**

**2026**

# **BAB I**

## **PENDAHULUAN**

### **1.1 Latar Belakang**

Pemrograman berorientasi objek merupakan salah satu konsep penting dalam pemrograman karena program dapat dibuat dengan mengelompokkan data dan fungsi ke dalam suatu objek. Beberapa konsep dasar dalam pemrograman berorientasi objek yaitu class, object, constructor, dan method. Dengan memahami konsep tersebut, programmer dapat membuat program yang lebih terstruktur dan mudah dikembangkan.

Pada praktikum ini, konsep tersebut diterapkan melalui studi kasus sederhana yaitu Mini Bank. Program Mini Bank digunakan untuk menggambarkan sebuah rekening yang memiliki data seperti nomor rekening, nama pemilik, dan saldo. Data tersebut dapat mengalami perubahan melalui beberapa proses seperti setor tunai, tarik tunai, dan pengecekan informasi rekening.

Melalui program ini, mahasiswa dapat memahami bagaimana sebuah class digunakan sebagai rancangan objek, constructor digunakan untuk memberikan nilai awal pada objek, serta method digunakan untuk melakukan suatu proses terhadap objek. Selain itu, praktikum ini juga menunjukkan bagaimana state atau keadaan suatu objek, khususnya saldo rekening, dapat berubah berdasarkan interaksi yang dilakukan. Dengan demikian, praktikum ini diharapkan dapat membantu mahasiswa memahami penerapan konsep dasar pemrograman berorientasi objek secara lebih nyata.

### **1.2 Tujuan**

Tujuan dari praktikum ini adalah:

1. Memahami konsep dasar pemrograman berorientasi objek, khususnya class, object, constructor, dan method.
2. Mengetahui cara membuat dan menggunakan object berdasarkan class yang telah dibuat.
3. Memahami penggunaan constructor untuk memberikan nilai awal pada object rekening.
4. Menerapkan method untuk melakukan berbagai operasi pada rekening, seperti setor tunai, tarik tunai, dan mengecek informasi rekening.
5. Memahami konsep interaksi objek dan manipulasi state melalui perubahan saldo rekening berdasarkan transaksi yang dilakukan.
6. Mampu menerapkan konsep pemrograman berorientasi objek dalam sebuah program sederhana menggunakan

## BAB II PEMBAHASAN

### 2.1 Rekening.java

```
package MiniBank;

public class Rekening {
    String nomorRekening;
    String namaPemilik;
    double saldo;
    double nominal;

    public Rekening(String nomor, String nama, double saldoAwal) {
        nomorRekening = nomor;
        namaPemilik = nama;
        saldo = saldoAwal;
        System.out.println("Rekening atas nama " + namaPemilik + " berhasil
dibuat dengan saldo Rp" + saldo);
    }

    public void setorTunai(double nominal) {
        if (nominal > 0) {
            saldo += nominal;
            System.out.println("Setor tunai Rp" + nominal + " berhasil.
Saldo saat ini: Rp" + saldo);
        } else {
            System.out.println("Gagal: Nominal setor harus lebih dari
0!");
        }
    }

    public void cekInformasi() {
        System.out.println("--- INFO REKENING ---");
        System.out.println("No. Rekening : " + nomorRekening);
        System.out.println("Nama Pemilik : " + namaPemilik);
        System.out.println("Saldo Akhir : " + saldo);
    }

    public void tarikTunai(double tarikTunai) {
        if (nominal < 10000) {
            System.out.println("Maaf transaksi gagal! Nominal penarikan
Rp10.000");
        } else if (nominal > saldo) {
            System.out.println("Maaf saldo anda tidak mencukupi! Saldo
anda Rp" + saldo);
        } else {
            saldo -= nominal;
            System.out.println("Penarikan berhasil.");
            System.out.println("Jumlah penarikan: Rp" + nominal);
            System.out.println("Saldo sekarang: Rp" + saldo);
        }
    }
}
```

- Class

```
public class Rekening {
```

➔ Rekening merupakan **class** yang menjadi rancangan untuk membuat objek rekening. Di dalam class ini terdapat data dan perilaku yang dimiliki sebuah rekening.

- **Attribute**

```
String nomorRekening;
```

```
String namaPemilik;
```

```
double saldo;
```

```
double nominal;
```

➔ Attribute adalah **data yang dimiliki oleh objek**.

- nomorRekening → menyimpan nomor rekening.
- namaPemilik → menyimpan nama pemilik rekening.
- saldo → menyimpan jumlah saldo.
- nominal → menyimpan nilai transaksi.

- **Constructor**

```
public Rekening(String nomor, String nama, double saldoAwal)
```

➔ Constructor digunakan untuk memberikan nilai awal saat objek dibuat.

```
    nomorRekening = nomor;
```

```
    namaPemilik = nama;
```

```
    saldo = saldoAwal;
```

Jadi ketika objek rekening dibuat, nomor rekening, nama pemilik, dan saldo awal langsung ditentukan.

- **Method setorTunai()**

```
public void setorTunai(double nominal)
```

➔ Method ini digunakan untuk melakukan setor tunai.

```
    saldo += nominal;
```

Artinya saldo akan bertambah sesuai nominal yang disetor.

- **Method cekInformasi()**

```
public void cekInformasi()
```

➔ Method ini digunakan untuk menampilkan informasi rekening, seperti nomor rekening, nama pemilik, dan saldo akhir.

Method ini tidak mengubah data atau state object, hanya menampilkan informasi.

- **Method tarikTunai()**

public void tarikTunai(double tarikTunai)

➔ Method ini digunakan untuk melakukan penarikan uang.

Sebelum menarik uang, program melakukan pengecekan:

- Nominal penarikan minimal Rp10.000.
- Nominal penarikan tidak boleh lebih besar dari saldo.

Jika transaksi berhasil:

saldo -= tarikTunai;

Saldo akan berkurang sesuai jumlah yang ditarik.

- **State dan Manipulasi State**

State adalah kondisi/data yang dimiliki oleh sebuah objek. Pada program ini, state utamanya adalah:

saldo

Pada program ini, objek rekening memiliki saldo awal yang diberikan melalui constructor. Saldo kemudian dapat berubah melalui method setorTunai() yang menambah saldo dan tarikTunai() yang mengurangi saldo. Perubahan tersebut merupakan bentuk manipulasi state pada objek rekening.

Jadi, manipulasi state terjadi ketika method setorTunai() dan tarikTunai() mengubah nilai saldo.

## 2.2 Main.java

```
package MiniBank;

import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        Rekening akunAktif = null;
        boolean isRunning = true;

        System.out.println("=== SISTEM PERBANKAN MINI ===");

        while (isRunning) {
            System.out.println("\nMenu Utama");
            System.out.println("1. Buka Rekening Baru");
            System.out.println("2. Setor Tunai");
            System.out.println("3. Tarik Tunai");
            System.out.println("4. Cek Informasi Rekening");
            System.out.println("0. Keluar");
            System.out.print("Pilih Menu: ");

            int pilihan = input.nextInt();
            input.nextLine(); // Membersihkan buffer enter
        }
    }
}
```

```

switch (pilihan) {
case 1:
    System.out.print("Masukkan No Rekening: ");
    String no = input.nextLine();
    System.out.print("Masukkan Nama Pemilik: ");
    String nama = input.nextLine();
    System.out.print("Masukkan Saldo Awal: ");
    double saldo = input.nextDouble();

    //Instansiasi Object / Menjalankan Constructor
    akunAktif = new Rekening(no, nama, saldo);
    break;

case 2:
    if (akunAktif == null) {
        System.out.println("Error: Mohon maaf, Anda belum
memiliki nomor rekening!");
    } else {
        System.out.print("Masukkan nominal setor: ");
        double setor = input.nextDouble();
        akunAktif.setorTunai(setor);
    }
    break;

case 3:
    if (akunAktif == null) {
        System.out.println("Error: Mohon maaf, Anda belum
memiliki nomor rekening!");
    } else {
        System.out.print("Masukkan nominal tarik tunai:
");
        double tarik = input.nextDouble();
        akunAktif.tarikTunai(tarik);
    }

case 4:
    if (akunAktif == null) {
        System.out.println("Error: Anda belum membuka
rekening!");
    } else {
        akunAktif.cekInformasi();
    }
    break;

case 0:
    isRunning = false;
    System.out.println("Sistem ditutup. Terima kasih!");
    break;

default:
    System.out.println("Pilihan tidak valid!");
}
}
input.close();
}
}

```

- Import Scanner

```
import java.util.Scanner;
```

➔ Digunakan untuk mengimport class Scanner yang berfungsi menerima input dari pengguna melalui keyboard.

- **Method main()**

```
public static void main(String[] args) {
```

➔ main() merupakan **method utama** yang pertama kali dijalankan ketika program Java dijalankan. Seluruh proses pada program Mini Bank dimulai dari method ini.

- **Membuat Object Scanner**

```
Scanner input = new Scanner(System.in);
```

➔ Baris ini membuat object input dari class Scanner.

Object tersebut digunakan untuk membaca input dari keyboard, seperti:

```
input.nextInt();
```

```
input.nextLine();
```

```
input.nextDouble();
```

- **Membuat variabel akunAktif**

```
Rekening akunAktif = null;
```

➔ Variabel akunAktif digunakan untuk menyimpan object rekening yang sedang digunakan.

Pada awal program nilainya:

**null**

yang berarti belum ada rekening yang dibuat.

Setelah pengguna memilih menu buka rekening:

```
akunAktif = new Rekening(no, nama, saldo);
```

maka akunAktif akan menunjuk ke object Rekening yang baru dibuat.

- **Variabel isRunning**

```
boolean isRunning = true;
```

Digunakan untuk menentukan apakah program masih berjalan atau tidak.

Nilai:

true → program tetap berjalan

false → program berhenti

Ketika pengguna memilih menu 0, nilainya diubah menjadi false.

- **Perulangan while**

```
while (isRunning) {
```

Digunakan agar menu utama terus ditampilkan selama program masih berjalan.

Jadi setelah pengguna selesai melakukan setor atau cek rekening, program akan kembali menampilkan menu.

- **Input pilihan menu**

```
int pilihan = input.nextInt();
```

Digunakan untuk membaca pilihan menu yang dimasukkan pengguna.

Misalnya pengguna memasukkan:

2

Maka: pilihan = 2

- **switch-case**

```
switch (pilihan) {
```

Digunakan untuk menentukan proses berdasarkan pilihan menu.

Program memiliki beberapa pilihan:

1 → Buka rekening

2 → Setor tunai

3 → Tarik tunai

4 → Cek informasi

0 → Keluar

Jadi switch akan menjalankan case yang sesuai dengan nilai pilihan.

- **Case 1: Membuka rekening**

case 1:

```
String no = input.nextLine();
```

```
String nama = input.nextLine();
```

```
double saldo = input.nextDouble();
```

```
akunAktif = new Rekening(no, nama, saldo);
```

```
break;
```

➔ Digunakan untuk membuat object Rekening menggunakan data yang dimasukkan pengguna.

- **Case 2: Setor tunai**

case 2:

```
double setor = input.nextDouble();
```

```
akunAktif.setorTunai(setor);
```

```
break;
```

➔ Digunakan untuk memanggil method `setorTunai()` agar saldo bertambah.

- **Case 3: Tarik Tunai**

case 3:

```
double tarik = input.nextDouble();
```

```
akunAktif.tarikTunai(tarik);
```

```
break;
```

➔ Digunakan untuk memanggil method `tarikTunai()` agar saldo berkurang sesuai nominal penarikan.

- **Case 4: Cek Informasi**

case 4:

```
akunAktif.cekInformasi();
```

```
break;
```

➔ Digunakan untuk memanggil method `cekInformasi()` dan menampilkan data rekening.

- **Case 0: Keluar**

case 0:

```
isRunning = false;
```

```
break;
```

➔ Digunakan untuk menghentikan perulangan dan keluar dari program.

- **Default**

default:

```
System.out.println("Pilihan tidak valid!");
```

➔ Digunakan untuk menangani pilihan menu yang tidak tersedia.

## 2.3 Tugas 1: Implementasi Fitur Tarik Tunai

```
package MiniBank;

public class Rekening {

    String nomorRekening;
    String namaPemilik;
    double saldo;

    public Rekening(String nomor, String nama, double saldoAwal) {

        nomorRekening = nomor;
        namaPemilik = nama;
        saldo = saldoAwal;
    }
}
```

```

        System.out.println("Rekening atas nama " + namaPemilik
            + " berhasil dibuat dengan saldo Rp" + saldo);
    }

    public void setorTunai(double nominal) {

        if (nominal > 0) {
            saldo += nominal;

            System.out.println("Setor tunai Rp" + nominal
                + " berhasil. Saldo saat ini: Rp" + saldo);
        } else {
            System.out.println("Gagal: Nominal setor harus lebih dari 0!");
        }
    }

    public void cekInformasi() {

        System.out.println("--- INFO REKENING ---");
        System.out.println("No. Rekening : " + nomorRekening);
        System.out.println("Nama Pemilik : " + namaPemilik);
        System.out.println("Saldo Akhir : " + saldo);
    }

    // Method tarik tunai
    public void tarikTunai(double nominal) {

        if (nominal < 10000) {

            System.out.println(
                "Transaksi Gagal: Minimal nominal penarikan Rp10000"
            );
        } else if (nominal > saldo) {

            System.out.println(
                "Transaksi Gagal: Saldo tidak mencukupi. Saldo Anda: Rp"
                + saldo
            );
        } else {

            saldo -= nominal;

            System.out.println("Penarikan berhasil.");
            System.out.println("Jumlah penarikan: Rp" + nominal);
            System.out.println("Saldo sekarang: Rp" + saldo);
        }
    }
}

```

- **Penjelasan:**

- **Method**

public void tarikTunai(double nominal)

→ Digunakan untuk melakukan penarikan uang.

Terdapat dua validasi:

if (nominal < 10000)

→ Memastikan penarikan minimal Rp10.000.

else if (nominal > saldo)

→ Memastikan nominal penarikan tidak lebih besar dari saldo.

Jika semua syarat terpenuhi:

saldo -= nominal;

→ Saldo akan berkurang. Ini merupakan **manipulasi state** pada object Rekening.

## 2.4 Tugas 2: Tantangan Multi-Akun

```
package MiniBank;

import java.util.Scanner;
import java.util.ArrayList;

public class Main {

    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        ArrayList<Rekening> daftarRekening = new ArrayList<>();
        Rekening akunAktif = null;
        boolean isRunning = true;
        System.out.println("=== SISTEM PERBANKAN MINI ===");
        while (isRunning) {

            System.out.println("\nMenu Utama");
            System.out.println("1. Buka Rekening Baru");
            System.out.println("2. Setor Tunai");
            System.out.println("3. Tarik Tunai");
            System.out.println("4. Cek Informasi Rekening");
            System.out.println("5. Ganti Akun");
            System.out.println("0. Keluar");
            System.out.print("Pilih Menu: ");

            int pilihan = input.nextInt();
            input.nextLine();

            switch (pilihan) {

                case 1:
                    System.out.print("Masukkan No Rekening: ");
                    String no = input.nextLine();

                    System.out.print("Masukkan Nama Pemilik: ");
                    String nama = input.nextLine();

                    System.out.print("Masukkan Saldo Awal: ");
```

```

        double saldo = input.nextDouble();
        Rekening rekeningBaru =
            new Rekening(no, nama, saldo);
        daftarRekening.add(rekeningBaru);
        akunAktif = rekeningBaru;
        System.out.println("Rekening berhasil ditambahkan.");
        break;

    case 2:
        if (akunAktif == null) {
            System.out.println("Error: Anda belum memiliki
rekening!");
        } else {
            System.out.print("Masukkan nominal setor: ");
            double setor = input.nextDouble();
            akunAktif.setorTunai(setor);
        }
        break;

    case 3:
        if (akunAktif == null) {
            System.out.println("Error: Anda belum memiliki
rekening!");
        } else {
            System.out.print("Masukkan nominal tarik tunai: ");
            double tarik = input.nextDouble();
            akunAktif.tarikTunai(tarik);
        }
        break;

    case 4:
        if (akunAktif == null) {
            System.out.println("Error: Anda belum membuka
rekening!");
        } else {
            akunAktif.cekInformasi();
        }
        break;

    case 5:
        if (daftarRekening.isEmpty()) {
            System.out.println("Belum ada rekening yang
tersedia.");
        } else {
            System.out.print(
                "Masukkan nomor rekening: "
            );
            String nomorCari = input.nextLine();
            boolean ditemukan = false;
            for (Rekening rekening : daftarRekening) {
                if (rekening.nomorRekening.equals(nomorCari)) {
                    akunAktif = rekening;
                    ditemukan = true;
                    System.out.println("Akun berhasil diganti ke "
+ rekening.namaPemilik);

```

```

        break;
    }
    }
    if (!ditemukan) {
        System.out.println("Rekening tidak ditemukan.");
    }
    }
    break;

    case 0:
        isRunning = false;
        System.out.println("Sistem ditutup. Terima kasih!");
        break;
    default:
        System.out.println("Pilihan tidak valid!");
    }
}
input.close();
}

```

- **Penjelasan:**

Karena tugas nya kita harus banyak rekening, maka kita ubah:

Rekening akunAktif = null;

Menjadi

ArrayList<Rekening> daftarRekening = new ArrayList<>();

Rekening akunAktif = null;

ArrayList digunakan sebagai tempat untuk menyimpan banyak object Rekening.

Lalu kita tambahkan import:

import java.util.ArrayList;

Setiap rekening baru yang dibuat akan dimasukkan ke dalam daftarRekening menggunakan add(). Selain itu, ditambahkan menu **Ganti Akun** untuk memilih rekening berdasarkan nomor rekening. Program mencari rekening menggunakan perulangan for-each, kemudian menjadikan rekening yang ditemukan sebagai akunAktif.

## **BAB III PENUTUP**

### **3.1 Kesimpulan**

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa konsep dasar Pemrograman Berorientasi Objek seperti class, object, constructor, dan method dapat diterapkan melalui program Mini Bank.

Program juga menunjukkan adanya manipulasi state melalui perubahan saldo saat melakukan setor dan tarik tunai. Pada Tugas 1 ditambahkan validasi penarikan, sedangkan pada Tugas 2 program dikembangkan agar dapat mengelola beberapa rekening menggunakan ArrayList dan fitur ganti akun. Dengan praktikum ini, konsep dasar PBO dapat dipahami dan diterapkan dalam program Java sederhana.